

Sistemas Paralelos - Examen Final - 26/03/2014

1. ¿Qué es un sistema paralelo? ¿Cuál es su importancia en la actualidad?
2. Demuestre la importancia de la memoria caché en las máquinas secuenciales con un ejemplo concreto. Que overhead pueden traer en memoria compartida que no ocurre en memoria distribuida.
3. Explique en qué consiste la programación con threads. ¿Que diferencia tiene con directivas? Explique ventajas y desventajas de cada uno.
4. Explique porqué es importante el balanceo de carga. Explique los balanceos de carga Estática Directa, Estática predictiva y Dinámica por demanda.
5. Explique pasaje de mensajes y que ventajas y desventajas tiene sobre memoria compartida.
6. Qué es el speedup, cuales son sus límites. Que pasa en las arquitecturas heterogéneas. Porque se puede dar un speedup superlineal.
7. Qué es la eficiencia, cuales son sus límites. Que pasa en las arquitecturas heterogéneas, ejemplifique.
8. Explique los siguientes paradigmas y ejemplifique: divide and conquer, pipelining, master/worker, SPMD. Indique y justifique a qué paradigma se adecúan más las siguientes redes estáticas: estrella, todos con todos, anillo, árbol.
9. Explique las descomposiciones en tareas: recursiva, basada en los datos, exploratoria, especulativa.

Respuestas

1. Un sistema paralelo es la combinación entre un algoritmo paralelo y una computadora paralela donde ejecutarlo. Los sistemas paralelos son muy importantes en la actualidad como forma para incrementar la velocidad de cómputo especialmente en actividades donde se requiere mucho procesamiento, aprovechar al máximo las capacidades del hardware y superar las limitaciones físicas del cómputo monoprocesador.
2. Consideremos un procesador con frecuencia de reloj de 1 GHz (1 ns clock) conectado a una memoria DRAM con una latencia de 100 ns (sin caches).

Supongamos que el procesador tiene dos unidades multiplicación-suma y es capaz de ejecutar 4 instrucciones por ciclo de reloj de 1 ns.

- El pico de rendimiento del procesador es 4 Gflops.
- Dado que la memoria tarda 100 ciclos de reloj en responder a un pedido, esto significa que 1 pedido a memoria significa 100 ciclos de espera para el procesador.

Si ahora sobre este procesador tenemos que multiplicar dos vectores de K elementos cada uno. Básicamente en cada producto se requiere dos accesos a memoria para buscar los operandos a multiplicar.

- Un producto asociado con dos lecturas, pone el ritmo de procesamiento en 1 operación de producto-suma cada 200 ns (es decir 2 operaciones de punto flotante cada 200 ns, una multiplicación y una suma).
- Pasamos de 4 Gflops pico a 10 Mflops reales.

Agregamos una memoria **caché** de 32 KB con latencia de 1 ns. Trataremos de multiplicar 2 matrices A y B de 32×32 (esto permitiría alojar en caché las matrices A , B y el resultado C).

- El fetch de las dos matrices para llevarlas de la caché al procesador significa leer 2K palabras:

$$2000 \times 100 \text{ ns} = 200 \text{ s}$$

- Multiplicar las dos matrices de $n \times n$ significa $2n^3$ operaciones. En nuestro problema, $n = 32$ significa 64K operaciones, que pueden realizarse en 16K ciclos (o 16 s) a 4 operaciones por ciclo de 1 ns:

$$\frac{64000}{4} \times 1 \text{ ns} = 16 \text{ s}$$

- Cada operación implica dos lecturas de caché. Aproximadamente:

$$64000 \times 2 \text{ ns} = 128 \text{ s}$$

- El tiempo total de cómputo pasa a ser entonces:

$$200 \text{ s} + 16 \text{ s} + 128 \text{ s} + 100 \text{ s (store resultado en memoria)} = 444 \text{ s}$$

- Esto corresponde a:

$$\frac{64000}{444 \text{ s}} = 144 \text{ Mflops}$$

El overhead en memoria compartida que no ocurre en distribuida, está relacionado con la coherencia de caché y false sharing.

3. Un thread es un único hilo de control en el flujo de un programa. Todos los threads tienen acceso a la memoria completa, sin embargo el stack de cada uno es tratado como local al thread, por una cuestión de vida del stack. Lo anterior implica que tenemos un modelo lógico de máquina con memoria global accesible por todos los threads y memoria local para los stacks. Da idea de arquitectura UMA para la memoria global. Si la memoria está distribuida, los costos de acceso a la memoria remota pueden ser mucho mayores que a la memoria local.

La programación con directivas provee paralelismo basado en threads, pero permite al programador abstraerse de las primitivas de bajo nivel que se usan al programar directamente con threads. Existen desventajas al utilizar directivas, el principal problema reside en que, al proveer transparencia y abstracción

siempre existe un impacto en la eficiencia por el overhead. En la programación con threads el intercambio de datos es explícito, esto ayuda a disminuir el overhead causado por movimiento de los datos, false sharing y memory contention, además la API es mucho más completa para condition waits, locks y provee mucha más flexibilidad para construir operaciones compuestas de sincronización que la programación con directivas. Finalmente, aunque no menos importante, el uso de la programación con threads está mucho más extendida por lo que existen muchas más herramientas y soporte que en la programación con directivas.

4. El balance de carga es importante para optimizar la performance. Si el balanceo no se hace bien, algunos procesadores pueden tener más carga que otros, haciendo que los procesadores con menos carga terminen antes que los otros, y queden ociosos, y así prolongar el tiempo total de ejecución.

El balance estático predictivo puede usarse cuando se conoce la cantidad de trabajo que va a realizarse, y la potencia de cálculo de los procesadores. De esta manera, se puede repartir el trabajo a priori entre los procesadores, de manera proporcional a su potencia.

El estático directo, en cambio, es cuando se reparten todas las tareas de manera equitativa (en una arquitectura homogénea).

En el balanceo de carga dinámico por demanda, se reparte una porción de trabajo estáticamente al comenzar el procesamiento, y a medida que los procesos van terminando su trabajo, se les asigna más.

5. El paradigma de programación con pasaje de mensajes tiene dos características claves:

1. Se asume que el espacio de direcciones se encuentra particionado.
2. Solo soporta paralelismo explícito.

Una máquina que soporta pasaje de mensaje consiste en p procesos cada uno con su espacio de direcciones exclusivo, esto implica que los datos se encuentran explícitamente particionados y que todas las interacciones requieren la cooperación entre dos procesos. El intercambio de mensajes no solo sirve para el intercambio de datos sino también para la sincronización entre procesos.

La principal diferencia entre el pasaje de mensajes y el paradigma de memoria compartida es que la comunicación es explícita. Una de las principales ventajas de este paradigma es que el programador es consciente del costo de las interacciones por lo que tenderá a tratar de minimizarlas al máximo, esto implica una mayor dificultad en la búsqueda de una solución pero con la ventaja de que se pueden alcanzar soluciones con un alto grado de paralelismo, y por lo tanto, performance.

6. El speedup es la medida que indica la relación entre el tiempo para resolver un algoritmo en un único procesador y el tiempo para resolver el mismo problema sobre una computadora paralela con p procesadores idénticos.

Se anota como $S = T_s/T_p$, donde T_s es el tiempo del algoritmo secuencial y T_p el tiempo del algoritmo paralelo.

En una arquitectura homogénea, el límite teórico para el speedup es p , con p procesadores iguales. Este límite se podría superar y tener un speedup superlineal cuando el algoritmo paralelo realiza menos trabajo que el algoritmo secuencial. Por ejemplo, en la búsqueda en un árbol binario, en cuanto uno de los procesadores encuentra el elemento, el trabajo termina sin recorrer todas las demás ramas. Otro motivo para el speedup superlineal puede ser que la solución paralela aproveche mejor el hardware. Por ejemplo, si los datos son muy grandes para la caché de un procesador, esto puede degradar la performance por el uso de memorias de menor jerarquía. Al particionar los datos en varios procesadores, los datos pueden caber en las respectivas cachés y así mejorar la tasa de hits y por ende la velocidad de acceso a los datos.

En arquitecturas heterogéneas, el speedup máximo ya no es p , sino que está dado por pct (potencia de cómputo total): la sumatoria de la relación de la potencia de cada procesador con respecto al procesador más potente:

$$pct = \sum_{i=0}^{p-1} pcr_i$$

$$pcr_i = \frac{potencia(procesador_i)}{potencia(procesador_{mejor})}$$

7. La eficiencia es una medida de la fracción de tiempo en la que cada procesador es empleado en forma útil en el algoritmo.

Se expresa como $E = S/p$ donde S es el speedup y p el número de procesadores. Lo óptimo es $E = 1$, cuando se aprovechan al máximo los p procesadores. Lo peor es $E = 0$, cuando el speedup se hace cero ya que el algoritmo no termina.

En las arquitecturas heterogéneas la eficiencia pasa a ser S/pct , con pct tal como se definió en el punto anterior para el speedup.

8. La estrategia divide and conquer descompone recursivamente un problema en subproblemas del mismo tipo hasta que este es lo suficientemente simple como para ser resuelto directamente. En general, las solución de los subproblemas son combinadas para solucionar el problemas original. Las redes basadas en árboles son las más adecuadas para este tipo de paradigma ya que esta topología refleja la descomposición recursiva de los problemas. Un nodo de procesamiento del árbol representará una porción del problema, teniendo como descendientes a nodos encargados de solucionar las particiones recursivas de su instancia del problema.

En el paradigma de pipeline un flujo de datos circula a través de una sucesión de procesos, donde cada uno realiza alguna tarea sobre estos. Un pipeline es una cadena de productores y consumidores, cada proceso en el pipeline puede ser visto como un consumidor de una secuencia de datos del proceso predecesor y un productor de datos para el proceso posterior en el pipeline. Además de la típica estructura lineal, los procesos se pueden organizar en el pipeline como arrays multidimensionales, árboles o grafos acíclicos.

El modelo *master-worker* o *master-slave* consiste en uno o más procesos master que generan/asignan trabajo a un conjunto de procesos worker. En general utilizan técnicas de mapeo dinámico centralizado donde a los procesos worker les es asignada una porción de trabajo pequeña y a medida que estas son completadas el proceso requiere nuevas tareas al manager. Al usar este tipo de esquema hay que tener especial cuidado en no causar un cuello de botella en el proceso master, lo cual puede suceder si las tareas son muy pequeñas por lo que la granularidad juega un rol central en este paradigma. También puede usarse un mapeo estático, donde el master va distribuyendo trabajo iterativamente. Las redes en estrella son la opción más atractiva. El paradigma master-worker se puede generalizar a una estructura jerarquizada en la cual un manager de más alto nivel asigna tareas a managers en niveles inferiores que a su vez subdividen la tarea en sus propios workers por lo que los arboles tambien son una buena alternativa.

Single Program Multiple Data (SPMD) es un modelo que se basa en múltiples procesadores autónomos ejecutando en simultáneo el mismo programa pero en puntos independientes.

9. La **descomposición recursiva** se suele aplicar a problemas tipo “divide y vencerás”. El problema básico es descompuesto en un conjunto de subproblemas independientes. A su vez estos son recursivamente descompuestos con el mismo criterio, hasta alcanzar la granularidad buscada. Esta estrategia resulta en una concurrencia natural, ya que los distintos subproblemas pueden resolverse concurrentemente.

La **descomposición basada en datos** se usa cuando se quiere traer concurrencia a algoritmos que operan sobre grandes conjuntos de datos.

La forma más común es basada en los datos de entrada. Cada tarea se asocia con una partición de los datos de entrada. Los procesadores pueden ser asignados a las tareas en un único nivel o reusarse en sucesivas divisiones a medida que el proceso avanza. Generalmente se requiere un paso posterior para “juntar” los resultados parciales.

Si los datos de *salida* pueden ser calculados independientemente entre sí, se puede descomponer en base a los datos de salida, como por ejemplo en la multiplicación de matrices.

En ocasiones se puede descomponer tanto en los datos de **entrada** y **salida**. Por ejemplo, en el tratamiento de imágenes, donde normalmente hay una sucesión de operaciones podemos tener una separación en tareas basada en el espacio de datos de entrada (las imágenes) y a su vez un análisis en función de las operaciones que definen las imágenes de salida.

Cuando los algoritmos hacen una serie de transformaciones sobre los datos, donde la salida de una etapa es la entrada de otra, se puede particionar en base a los datos **intermedios**.

Con la **descomposición exploratoria**, la descomposición del problema va evolucionando durante la ejecución. Esto suele ocurrir en algoritmos de búsqueda. Se divide el espacio de búsqueda en partes más chicas, se procesan concurrentemente, hasta que se encuentran las soluciones. Puede generar speedup superlineal.

La **descomposición especulativa**, se usa cuando no se conocen a priori las dependencias que habrá entre las tareas, y no podemos asegurar de que las descomposiciones que haríamos de antemano sean independientes. Se intenta predecir cómo van a evolucionar las tareas. Hay un compromiso entre concurrencia alcanzable y porcentaje de procesos que hay que abortar y reiniciar.